SBE DevOps

Container Security Hardening

Architecture, Tooling, and Audit Trail

Version 1.0 | May 2026 | sbe-devops/security-actions

This document describes the container security controls built into SBE DevOps CI/CD pipelines. Every container image released from an SBE project passes through a defined security gate before reaching production. The components, rationale, and audit evidence produced are documented here for internal reference and external audit.

1. Security Philosophy

Shift Left, Gate Right

Security is applied at every stage of the container lifecycle — not bolted on after the fact. Vulnerabilities are caught before images reach any registry. SBOMs are generated before push. Images are signed and attested after push so consumers can cryptographically verify provenance. No image enters production without passing all gates.

Core Principles

- **No secrets in images** — credentials are injected at runtime via AWS SSM, never baked in.
- **Non-root by default** — all images run as UID 1001 (appuser), never root.
- **Immutable releases** — ECR repositories use IMMUTABLE_WITH_EXCLUSION; released tags cannot be overwritten.
- **Fail on CRITICAL CVEs** — builds fail before push if HIGH or CRITICAL vulnerabilities are found.
- **Verifiable provenance** — every released image is signed by digest and carries an attested SBOM.
- **Arm64 only** — Graviton reduces attack surface by eliminating x86-specific exploit vectors.

2. Pipeline Architecture

The security pipeline is embedded in the container release workflow (sbe-devops/container-workflows). Security actions are sourced from sbe-devops/security-actions and versioned independently.

Release Pipeline Flow

Stage	Action	Runs On	Blocks Push?
1. Build	docker/build-push-action (load)	Local daemon (TEST_IMAGE)	N/A
2. Version Extract	docker run --entrypoint ''	TEST_IMAGE	Yes — fails if no semver
3. CVE Scan	security-actions/scan (Trivy)	TEST_IMAGE	Yes — fails on HIGH/CRITICAL
4. SBOM Generation	security-actions/sbom (Syft)	TEST_IMAGE	No — generates artifact
5. Structure Tests	container-structure-test	TEST_IMAGE	Yes — fails on test error
6. Push to ECR	docker/build-push-action (push)	ECR registry	N/A
7. Sign Image	security-actions/sign (Cosign)	ECR image by digest	No — post-push
8. Attest SBOM	security-actions/attest (Cosign)	ECR image by digest	No — post-push

Stages 1-5 run against the locally loaded TEST_IMAGE — no vulnerable image ever touches ECR. Stages 7-8 run post-push using the immutable image digest, ensuring the signature and attestation reference exactly what was pushed.

3. CVE Scanning — Trivy

Trivy (by Aqua Security) is an open-source vulnerability scanner. It scans the container image filesystem and package manifests against the OSV, NVD, and OS-vendor advisory databases.

What Trivy Scans

OS packages — RPM (Amazon Linux), DEB, APK — checks installed versions against CVE databases.

Language packages — Python (pip), Node (npm), Go modules, Java (Maven/Gradle), Ruby (gems).

Secrets — Detects accidentally embedded API keys, tokens, and credentials.

Misconfigurations — Dockerfile best practice violations (optional, not enabled by default).

Configuration

Input	Setting	Rationale
Severity threshold	HIGH, CRITICAL	LOW/MEDIUM are noise in base images; actionable findings only
Exit code	1 (fail build)	Vulnerable images never reach ECR
ignore-unfixed	true	Eliminates CVEs with no available fix — reduces alert fatigue
Output format	SARIF	Uploaded to GitHub Security tab for visibility and tracking
Scan target	TEST_IMAGE (local)	Scanned before push — registry stays clean

SARIF Output

Trivy results are written in SARIF (Static Analysis Results Interchange Format) and uploaded to the GitHub Security tab via codeql-action/upload-sarif. This provides a persistent, searchable audit trail of every scan result — including historical findings — visible to all repo contributors with Security permission.

4. Software Bill of Materials — Syft

A Software Bill of Materials (SBOM) is a machine-readable inventory of every software component in a container image — OS packages, language libraries, and their versions. Syft (by Anchore) generates the SBOM from the built image before push.

Why SBOMs Matter

- Incident response — when a new CVE drops (e.g. Log4Shell), you can immediately query which images are affected without re-scanning everything.
- Compliance — US Executive Order 14028 (2021) mandates SBOMs for software sold to federal agencies. Ahead of the curve.
- Supply chain visibility — shows exactly what third-party code is running in production, including transitive dependencies.
- Audit trail — SBOM attested to the image digest proves the image content has not changed since the SBOM was generated.

Format

SBOMs are generated in **CycloneDX JSON** format — the OWASP-maintained standard designed specifically for security use cases. CycloneDX is the format required by most compliance frameworks and supported natively by Trivy, Grype, and major SCA tools. SPDX JSON is also available as an option.

The SBOM is generated from the local TEST_IMAGE before push, then attached as a Cosign attestation to the pushed image by digest. Anyone who pulls the image can verify and retrieve the SBOM using: cosign download attestation @

5. Image Signing — Cosign + Sigstore

Container images are signed using Cosign with keyless signing via the Sigstore public infrastructure. Keyless signing eliminates the operational overhead of managing long-lived signing keys — the identity proof comes from the GitHub Actions OIDC token.

How Keyless Signing Works

1. **OIDC token** — GitHub Actions provides a short-lived OIDC token proving the workflow identity (e.g. sbe-devops/csi-python, release workflow, main branch).
2. **Fulcio CA** — Sigstore's Fulcio certificate authority exchanges the OIDC token for a short-lived X.509 signing certificate tied to the workflow identity.
3. **Sign by digest** — Cosign signs the image digest (sha256:...) using the ephemeral certificate. The signature is pushed to the ECR repository as an OCI artifact.
4. **Rekor log** — The signing event is recorded in Sigstore's Rekor transparency log — a tamper-evident, append-only ledger. The log entry is publicly auditable.
5. **Certificate expires** — The signing certificate expires in minutes. No long-lived key exists to be stolen or rotated.

Verification

Anyone with Cosign installed can verify a signed image:

```
cosign verify --certificate-identity-regexp 'https://github.com/sbe-devops/.*'
--certificate-oidc-issuer 'https://token.actions.githubusercontent.com' <image>@<digest>
```

Signing is by digest, not by tag. Tags are mutable; digests are not. A valid signature on a digest proves the exact image bytes were produced by our pipeline — not just something with the same tag name.

6. SBOM Attestation — Cosign Attest

Attestation binds the SBOM to the image digest using the same Sigstore infrastructure as image signing. The SBOM is stored as an OCI artifact in ECR alongside the image, retrievable by anyone with pull access.

Attestation vs. Signing

	Image Signing	SBOM Attestation
What it proves	Image bytes are authentic and pipeline-produced	SBOM accurately describes the image contents at release time
What is signed	The image digest	The SBOM + image digest together
Storage	OCI artifact in ECR alongside the image	OCI artifact in ECR alongside the image
Retrieval	cosign verify @	cosign download attestation @

7. Tooling Reference

Tool	Version Pinned	Purpose	Maintained By
Trivy	aquasecurity/trivy-action@v0.36.0	CVE scanning, SARIF output	Aqua Security
Syft	anchore/sbom-action@v0	SBOM generation (CycloneDX/SPDX)	Anchore
Cosign	sigstore/cosign-installer@v3	Image signing + SBOM attestation	Sigstore / OpenSSF
CodeQL Upload	codeql-action/upload-sarif@v4	SARIF -> GitHub Security tab	GitHub
container-structure-test	v1.22.1 (binary)	Image structure validation	Google
Fulcio CA	Public Sigstore instance	Short-lived signing certificates	Sigstore / Linux Foundation
Rekor	Public Sigstore instance	Signing transparency log	Sigstore / Linux Foundation

8. Audit Trail — What Every Release Produces

Every image released through the SBE container pipeline produces a complete, verifiable audit trail. The following artifacts are available for any released image.

Artifact	Where It Lives	What It Proves
SARIF scan report	GitHub Security tab (repo)	CVE state at build time — findings, severity, fix availability
SBOM (CycloneDX JSON)	ECR — OCI artifact on image digest	Complete component inventory at release time
Cosign signature	ECR — OCI artifact on image digest	Image bytes produced by our GitHub Actions pipeline
SBOM attestation	ECR — OCI artifact on image digest	SBOM is bound to this exact image digest
Rekor log entry	Sigstore public transparency log	Tamper-evident record of signing event, timestamp, identity

GHA workflow run	GitHub Actions run history	Full pipeline execution log, steps, inputs, runner
Build metadata file	Baked into image at /usr/libexec/*/build	Runtime + calver tag visible in container stdout at startup

```
To retrieve all security artifacts for a released image: cosign verify @ # verify signature cosign download attestation @ # retrieve SBOM cosign verify-attestation --type cyclonedx @ # verify + decode SBOM
```

9. References

Resource	URL
Trivy documentation	https://aquasecurity.github.io/trivy/
Trivy GitHub Action	https://github.com/aquasecurity/trivy-action
Syft SBOM generator	https://github.com/anchore/syft
anchore/sbom-action	https://github.com/anchore/sbom-action
Cosign	https://github.com/sigstore/cosign
Sigstore project	https://www.sigstore.dev
Fulcio CA	https://github.com/sigstore/fulcio
Rekor transparency log	https://github.com/sigstore/rekor
CycloneDX specification	https://cyclonedx.org/specification/overview/
SPDX specification	https://spdx.dev/specifications/
SARIF specification	https://docs.github.com/en/code-security/code-scanning/integrating-with-code-scanning/sarif-support-for-code-scanning
OCI image spec (CNCF)	https://github.com/opencontainers/image-spec
OpenSSF Scorecard	https://securityscorecards.dev
US EO 14028 — SBOM mandate	https://www.cisa.gov/sbom
NIST SP 800-190 — Container Security	https://csrc.nist.gov/publications/detail/sp/800-190/final